LT 6
Data type - Tuples

What is a Tuple?

A tuple is a sequence of immutable Python objects.

`tup1 = ()` is an empty tuple

`tup2 = (1,)` contains only one element in the tuple. Not `(1)` !!!

```
>>> type((1,))
>>> type((1))
```

Why use a tuple instead of a list?

- Program execution is faster when manipulating a tuple than a list.
- When the data is not to be modified.

Indexing and Slicing

```
tup = ('a', 'b', 'c', 'd',
'e', 'f', 'g')

tup[2] = 'c'
tup[2:5] = ('c', 'd', 'e')
```

Tuples are immutable.

```
tup = ('a', 'b', 'c', 'd',
'e', 'f', 'g')
```

Cannot update a value in the tuple by assignment

```
tup[2] = 'h'
```

Tuples are immutable.

```
tup = ('a', 'b', 'c', 'd',
'e', 'f', 'g')
```

Cannot add any element into a tuple

```
tup.append('h')
```

Tuples are immutable.

```
tup = ('a', 'b', 'c', 'd',
'e', 'f', 'g')
```

Cannot remove any element in a tuple

```
del tup[6]
tup.remove('g')
```

Tuples are immutable.

```
tup = ('a', 'b', 'c', 'd',
'e', 'f', 'g')
```

Can delete the whole tuple

```
del tup
```

Concatenation +

```
tup1 = ('a', 'b')
>>> id(tup1)

>>> tup1 = tup1 + (1, 2)
>>> tup1
>>> id(tup1)
```

A new tuple is created !

Repetition *

```
tup = ('a', 'b', 'c')

>>> tup * 3
('a', 'b', 'c', 'a', 'b',
'c', 'a', 'b', 'c')
```

Membership

```
tup = ('a', 'b', 'c')

>>> 'b' in tup
True

>>> 'z' in tup
False
```

Iteration

```
tup = ('a', 'b', 'c')

for ele in tup:
    print(ele)

for i in range(len(tup)):
    print(tup[i])
```

```
'a'
'b'
'c'
```

Python's Built-in Functions

```
string = 'aAbz'

>>> tup = tuple(string)
tup = ('a', 'A', 'b', 'z')
                              #ord('a')=97
>>> len(tup) == 4             #ord('b')=98
>>> max(tup) == 'z'           #ord('z')=122
>>> min(tup) == 'A'           #ord('A')=65
```

Python's Built-in Functions

```
tup = (5, 2, 3)

>>> id(tup)     2121760223304

>>> sorted(tup)
>>> tup          [2, 3, 5]
>>> id(tup)     2121760223304
```

It became a list!

Return values as a Tuple

```
def test();
    a, b = 1, 2
    return a, b

>>> test()
```

The output will be a tuple.

Use of Tuples

Function can only return a single value, but by making that value a tuple, we can effectively group together as many values as we like, and return them together.

```
tup = (1,2,3,4,5)

def score(tup):
    high = max(tup)
    low = min(tup)
return (high, low)
```

With or without the bracket, the output will still be a tuple.

Use of Tuples

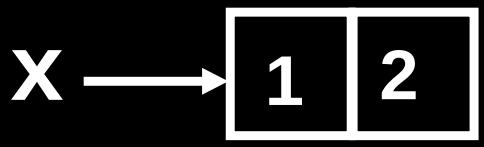
For example, we could write a function that returns both the area and the circumference of a circle of radius r:

```
from math import *

def f(r):
    c = 2 * pi * r
    a = pi * r * r
    return c, a
```

Box-and-Pointer

```
x = (1, 2)
```




```
>>> x → (1, 2)
```

This is a wrong way to visualize a tuple !!!

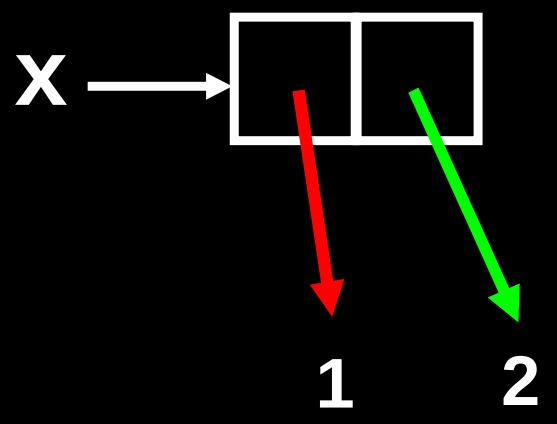
- The values 1 and 2 are not stored inside the tuple, they are stored ***outside*** at some memory addresses.

Box-and-Pointer

```
x = (1, 2)

>>> x → (1, 2)

>>> x[0] → 1
>>> x[1] → 2
```



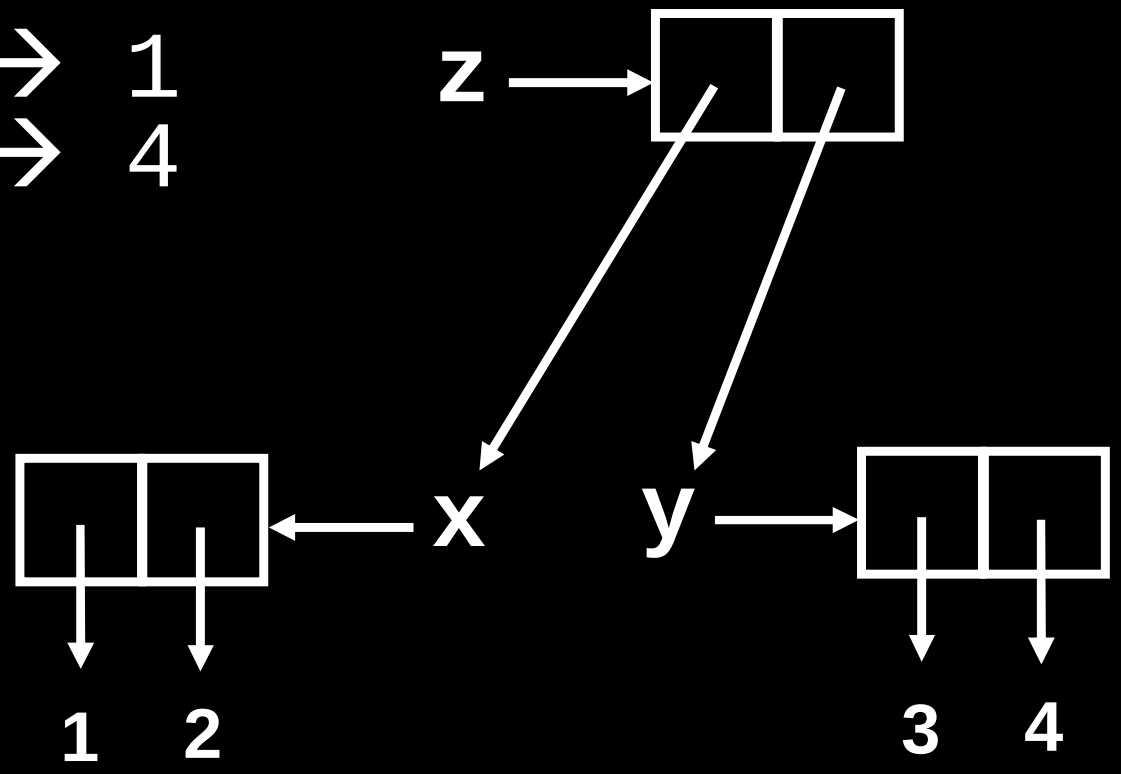
- Variable x points to the tuple
- Left pointer x[0] points to a memory storing the number **1**
- Right pointer x[1] points to a memory storing the number **2**

Box-and-Pointer

```
x = (1, 2)
y = (3, 4)

z = (x, y)  # z is tuple of tuples

>>> z[0][0] → 1
>>> z[1][1] → 4
```

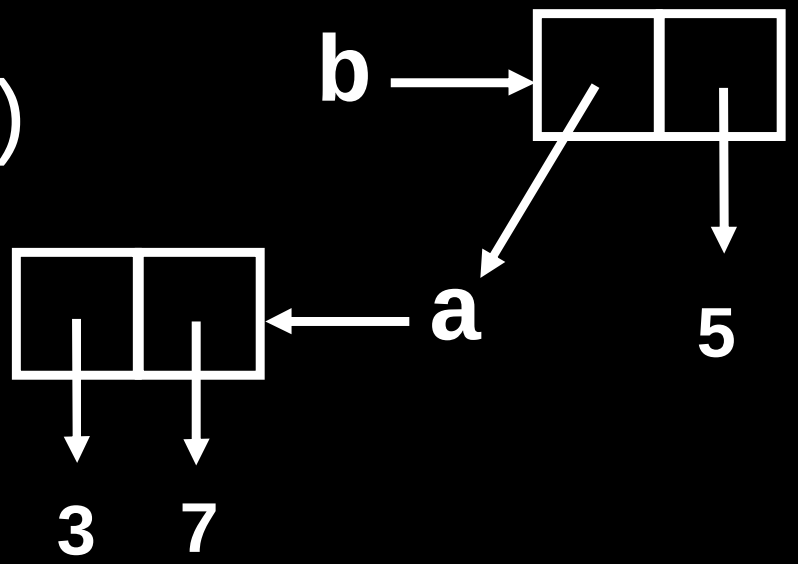


Tuple of Tuples

```
a = (3, 7)
>>> a → (3, 7)

b = (a, 5)
>>> b → ((3, 7), 5)

>>> b[0][0] → 3
>>> b[0][1] → 7
>>> b[1]    → 5
```



Equality

What does equality mean?

Two possibilities

1. Identity / Identical

This means the two objects are EXACTLY THE SAME, i.e. they are identical.

In Python, we use **`is`** to check for identity.

For example,

```
a = 5
b = a              # assign a to b

>>>  a is b    # True
```

Two possibilities

2. Equivalence

This means two objects have the same value, even though they may not be the same object.

In Python, we use **==** to test for equivalence.

```
a, b = 5, 5
>>> a == b      # True
>>> a is b      # False
```

Equivalence is not the same as Identical.

Equality

`is` returns **`True`**, the two objects are identical.

`==` returns **`True`**, the two objects have the same value.

```
a = 1
b = 2
x = (a, b)
y = (a, b)
>>> x is y → False
>>> x == y → True
```

```
z = x
>>> z is x → True
>>> z is y → False
>>> z == y → True
```

Be Careful with **`is`**

In Python, **`is`** cannot be used to compare numbers reliably.

```
>>> 3 is 3
```

⇒ **`True`**

```
>>> 3.000 is 3
```

⇒ **`False`**

Summary

Use == when you are comparing the two values.